

토큰 회전과 자동 재발급

학습 체크리스트

☐ Refresh Token을 한 번만 쓰고 교체하는 이유를 설명할 수 있다.

☐ 이미 사용된 토큰이 다시 들어올 때의 대응을 이해할 수 있다.

☐ 회전이 적용된 재발급 API를 작성할 수 있다.

☐ 로그아웃한 Access Token을 즉시 차단할 수 있다.

☐ 401 응답에 반응하는 자동 재발급을 구현할 수 있다.

학습 범위와 선수 조건

- 선수 지식: Access/Refresh 토큰 분리, Redis 저장소, HttpOnly 쿠키 로그인
- 핵심 질문: 탈취된 토큰을 어떻게 알아채고 막을 것인가
- 기준 환경: Spring Boot 4 · Spring Security 7 계열
- 완성 목표: 로그인부터 로그아웃까지 인증 흐름을 완성

전체 진행 순서



토큰 회전과 재사용 감지

수명이 긴 토큰이 위험한 이유

- 긴 유효기간: Refresh Token은 보통 14일 정도로 길게 잡음
- 탈취 시: 유출된 토큰으로 14일 내내 재발급이 가능
- 구분 불가: 서버는 정상 사용자와 탈취자의 요청을 구분하지 못함

한 번만 쓰이는 토큰으로 바꾸기

- 회전(Rotation): 재발급할 때마다 사용한 토큰을 폐기하고 새로 발급
- 한 번만 사용: 하나의 Refresh Token은 오직 한 번만 쓸 수 있음
- 효과: 탈취되어도 다음 재발급 시점부터는 사용할 수 없음

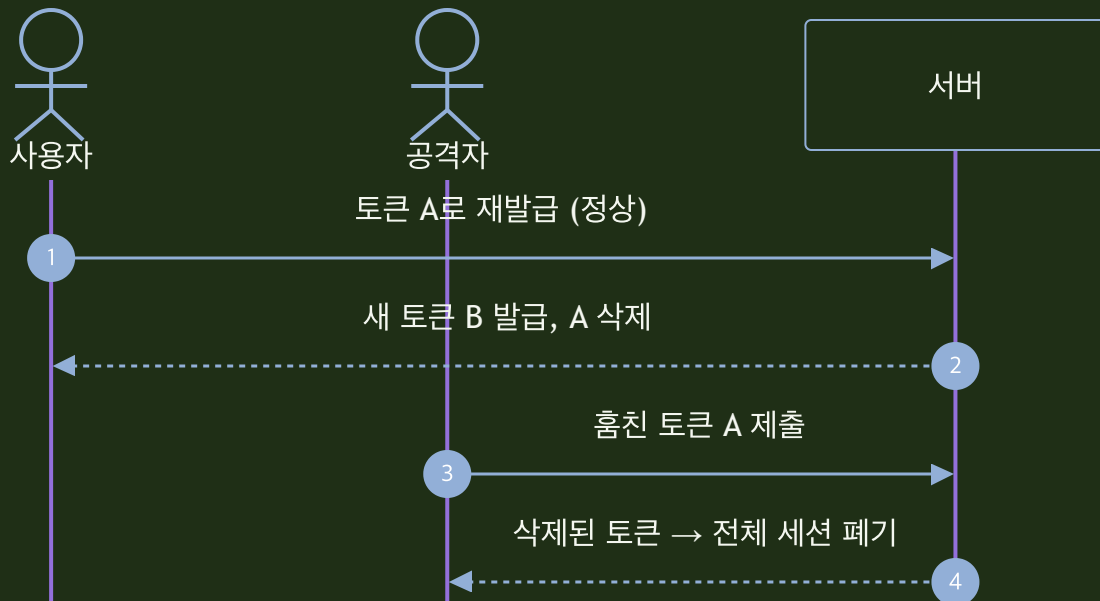
회전 처리 흐름



이미 폐기된 토큰이 다시 들어온다면

- 정상 상황: 사용이 끝난 토큰은 다시 제출될 이유가 없음
- 재사용 감지: 그런 요청이 들어오면 토큰이 유출된 것으로 판단
- 대응: 해당 사용자의 Refresh Token을 모두 삭제하고 재로그인을 요구

토큰 유출 상황의 처리



핵심 용어 정리

토큰 회전 (Token Rotation)

재발급할 때마다 기존 Refresh Token을 폐기하고 새 토큰으로 교체하는 방식

재사용 감지 (Reuse Detection)

이미 사용된 토큰이 다시 제출되면 유출로 판단해 해당 사용자의 토큰을 모두 폐기하는 처리

재발급 API 구현

POST /api/auth/refresh 처리 순서

- 쿠키 읽기: 요청에 담긴 `refresh_token` 쿠키 값을 가져옴
- 토큰 검증: 서명이 올바른지, `type` 이 `refresh` 인지 확인
- Redis 확인: 저장된 `jti` 가 아직 남아 있는지 조회
- 회전·재발급: 기존 토큰을 삭제하고 새 토큰 한 쌍을 발급

재사용 감지와 회전 처리

```
1  if (!refreshTokenRepository.existsById(jti)) {  
2      refreshTokenRepository.deleteAllByUsername(username);  
3      throw new BadCredentialsException("토큰 재사용 감지");  
4  }  
5  
6  refreshTokenRepository.deleteById(jti); // 회전: 쓴 토큰 폐기
```

- 저장소에 없으면 유출로 판단하고, 있으면 지운 뒤 새로 발급

새 토큰 한 쌍을 발급하고 저장

```
1 String newAccess = jwtTokenProvider.createAccessToken(username);
2 String newRefresh = jwtTokenProvider.createRefreshToken(username);
3
4 refreshTokenRepository.save(
5     RefreshToken.of(newRefresh, username));
```

- 새 Refresh Token의 `jti`도 Redis에 저장해 다음 회전에 대비

재발급 컨트롤러

```
1  @PostMapping("/refresh")
2  public ResponseEntity<TokenResponse> refresh(
3      @CookieValue("refresh_token") String refreshToken) {
4      TokenPair pair = authService.reissue(refreshToken);
5      ResponseCookie cookie = authCookieFactory.create(pair.refreshToken());
6
7      return ResponseEntity.ok()
8          .header(HttpHeaders.SET_COOKIE, cookie.toString())
9          .body(new TokenResponse(pair.accessToken()));
10 }
```

- 쿠키로 받고, 새 쿠키와 새 Access Token을 함께 내려줌

핵심 용어 정리

@CookieValue

요청에 담긴 쿠키 값을 컨트롤러 메서드의 파라미터로 바로 받아 오는 애너테이션

로그아웃과 블랙리스트

로그아웃해도 남아 있는 토큰

- Refresh만 삭제: 로그아웃 시 Redis의 Refresh Token만 지워서는 부족
- Access는 살아 있음: 이미 발급된 Access Token은 만료 전까지 통과
- 공백 구간: 로그아웃했는데 최대 15분간 요청이 인증되는 상태

블랙리스트로 즉시 차단하기

- 등록: 로그아웃 시 Access Token의 `jti`를 Redis에 기록
- 확인: 이후 요청마다 필터에서 블랙리스트에 있는지 조회
- TTL은 남은 시간만큼: 토큰이 만료된 이후에는 보관할 필요가 없음

차단 대상을 저장하는 엔티티

```
1  @RedisHash("blacklist")
2  public class BlacklistToken {
3
4      @Id
5      private String jti;
6
7      @TimeToLive
8      private Long ttl;
9  }
```

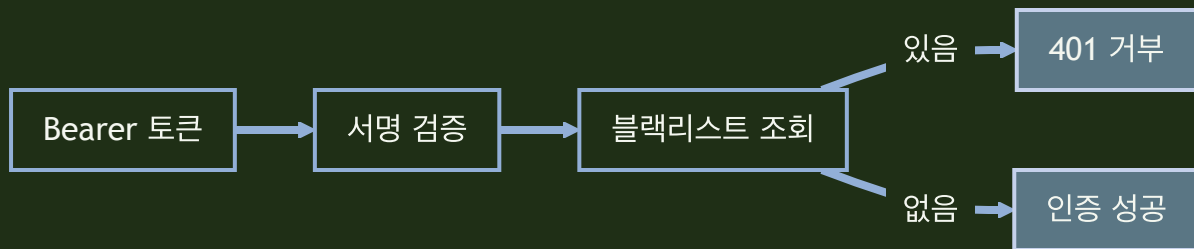
- 저장소는 `CrudRepository` 만 상속하면 `existsById` 로 조회 가능

로그아웃 시 두 토큰을 함께 정리

```
1 public void logout(String accessToken, String refreshToken) {  
2     Claims refresh = jwtTokenProvider.parseClaims(refreshToken);  
3     refreshTokenRepository.deleteById(refresh.getId());  
4  
5     Claims access = jwtTokenProvider.parseClaims(accessToken);  
6     long ttl = jwtTokenProvider.getRemainingSeconds(access);  
7     blacklistTokenRepository.save(new BlacklistToken(access.getId(), ttl));  
8 }
```

- Refresh는 삭제하고, Access는 남은 시간만큼 블랙리스트에 등록

로그아웃 이후 요청 처리



핵심 용어 정리

블랙리스트 (Blacklist)

아직 만료되지 않았지만 더 이상 인증에 사용할 수 없도록 등록해 둔 토큰 목록

자동 재발급

재발급 경로는 인증 없이 열어 두기

```
1 http.authorizeHttpRequests(auth → auth
2     .requestMatchers(
3         "/api/auth/login",
4         "/api/auth/refresh",
5         "/api/auth/logout"
6     ).permitAll()
7     .anyRequest().authenticated()
8 );
```

- Access Token이 만료된 상태에서 호출하는 경로이므로 인증을 요구하면 안 됨

쿠키를 주고받기 위한 CORS 설정

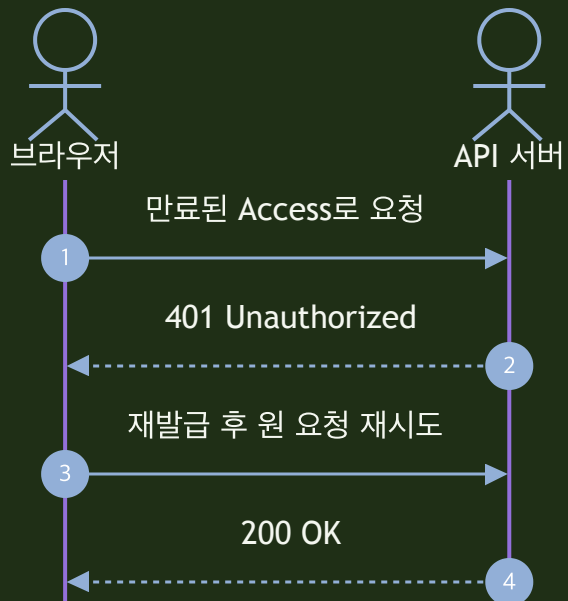
```
1 CorsConfiguration config = new CorsConfiguration();
2 config.setAllowedOriginPatterns(List.of("http://localhost:5173"));
3 config.setAllowedMethods(List.of("GET", "POST", "DELETE"));
4 config.setAllowCredentials(true);
```

- 쿠키를 함께 보내려면 `allowCredentials(true)` 설정이 반드시 필요

자동 재발급(Silent Refresh)이란

- 동작: 401 응답을 받으면 곧바로 `/api/auth/refresh` 를 호출
- 재시도: 새 Access Token을 받아 원래 요청을 한 번 다시 보냄
- 사용자 경험: 로그인 화면으로 이동하지 않고 작업이 그대로 이어짐

자동 재발급 흐름



401을 가로채는 요청 함수

```
1  async function authorizedFetch(url, options = {}, retry = true) {
2    const res = await fetch(url, { ...options, credentials: 'include' });
3    if (res.status === 401 && retry) {
4      await refreshAccessToken();
5      return authorizedFetch(url, options, false);
6    }
7    return res;
8  }
```

- `refreshAccessToken()` 은 재발급 API를 부르는 함수이며, 재시도는 한 번만 수행

동작 검증

검증 시나리오

시나리오	기대 결과
로그인	Access 발급, Refresh 쿠키 저장
만료 후 API 호출	자동 재발급 후 요청 성공
로그아웃	쿠키 삭제, 204
로그아웃 후 호출	블랙리스트 차단, 401

자주 발생하는 문제

증상	해결
쿠키가 전송되지 않음	<code>credentials: 'include'</code> 와 <code>allowCredentials</code> 설정 확인
로컬에서 쿠키가 저장되지 않음	로컬 환경에서는 <code>cookie-secure</code> 를 해제
재발급이 401을 반환	인가 설정에 <code>/api/auth/refresh</code> 추가

학습 요약

- 회전: 사용한 Refresh Token은 폐기하고 새 토큰으로 교체
- 재사용 감지: 이미 사용된 토큰이 들어오면 해당 사용자의 토큰을 모두 폐기
- 블랙리스트: 로그아웃한 Access Token을 남은 시간만큼 차단
- 자동 재발급: 401을 만나면 갱신 후 원래 요청을 한 번 재시도

예상 질문과 답변

Q&A: 구현

- Q. 재사용 감지는 왜 해당 사용자의 토큰을 모두 지우나요?
- A. 서버는 정상 사용자와 공격자를 구분할 수 없어서, 모두 로그아웃시키고 다시 로그인하게 하는 편이 안전합니다.
- Q. 로그아웃할 때 Refresh Token만 지우면 안 되나요?
- A. 이미 발급된 Access Token이 만료 전까지 통과하므로, 블랙리스트까지 등록해야 바로 막힙니다.

Q&A: 자동 재발급

- Q. 여러 요청이 동시에 401을 받으면 어떻게 되나요?
- A. 각자 재발급을 시도하면 두 번째부터는 이미 폐기된 토큰을 쓰게 되어 재사용 감지가 오작동합니다. 재발급 요청은 하나로 묶어 한 번만 보내야 합니다.
- Q. 여러 기기에서 로그인하면 재사용 감지에 걸리나요?
- A. 기기마다 다른 토큰이 발급되므로 문제없습니다. 같은 토큰이 두 번 쓰일 때만 감지됩니다.

Q&A: 성능

- Q. 매 요청마다 블랙리스트를 조회하면 느려지지 않나요?
- A. `jti` 하나를 확인하는 단순 조회라 부담이 적습니다.
- Q. 재발급 요청이 자주 일어나지 않나요?
- A. Access Token 수명(예: 15분)에 한 번 정도라 빈도가 높지 않습니다.

Q&A: 실무

- Q. Access Token 수명은 보통 얼마로 잡나요?
- A. 15~30분 사이가 일반적이며, 서비스 성격에 따라 조정합니다.
- Q. 모바일 앱에서도 쿠키를 써야 하나요?
- A. 앱은 안전한 자체 저장소를 쓸 수 있어, 보통 헤더로 Refresh Token을 주고받습니다.